



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

CRIPTOGRAFÍA Y SEGURIDAD

PRÁCTICA 2: PENTESTING: RECOPILACIÓN DE INFORMACIÓN

Arroyo Ramírez Ian Alejandro

PROFESOR:

María Ximena Lezama Hernández

AYUDANTES:

Adrián Felipe Fernández Romero

Alejandro Jácome Delgado

FECHA DE ENTREGA:

1 de septiembre de 2026



Actividad y Entregables

En esta sección se detallan los entregables correspondientes al análisis de los dominios mediante el script en bash. Los dominios a analizar son: `unam.mx`, `pemex.com`, `gob.mx`, `ipn.mx` y una página a elección.

Requisitos

Dado que buscamos encontrar vulnerabilidades en los sistemas, requerimos recopilar la mayor cantidad de información disponible que nos proporcione una visión general de la infraestructura del sistema. De esta manera, en la práctica obtenemos información de conectividad y rendimiento, información legal y de registro, mapeo de direcciones IP y segmentos de red, la topología de tráfico y enrutamiento, la enumeración completa de DNS y el mapeo de subdominios, así como la información del dominio asociado al certificado SSL/TLS y las cabeceras HTTP de redirección en el caso de que usemos IPs para poder hacer el análisis completo con el script utilizando IPs como entrada, pues algunas herramientas son sensibles a utilizar únicamente nombres de dominio. Con el análisis de esta información podemos llegar a encontrar puntos débiles de los dominios o IPs para que, a partir de una búsqueda exhaustiva de vulnerabilidades en bases de datos oficiales, podamos en algún momento informar a las autoridades correspondientes (la información para estos reportes también se incluye en la búsqueda del script) para que se puedan implementar soluciones.

Identificación de fuentes de información

El uso de las herramientas `ping`, `nslookup`, `tracert`, `whois`, `sublist3r`, `subfinder`, `findomain`, `dnsmap`, `dnsrecon` y `nmap` nos permite recopilar información veraz y pública sobre la infraestructura de los servicios que queremos analizar.

A partir de la información obtenida, podemos contrastarla con bases de datos oficiales de vulnerabilidades como la *National Vulnerability Database* (NVD) [1], la lista de Common Vulnerabilities and Exposures (CVE) [2] y la Exploit Database [3] para buscar fallos de seguridad conocidos según los servicios y versiones detectadas. Por otro lado, la propiedad de los dominios, los bloques de red y la asignación de IPs se puede verificar en fuentes y registros oficiales como LACNIC [4], IANA [5] y NIC México [6]. Además, la validez técnica de los registros de correo y seguridad que analizamos (como SPF, DMARC o DNSSEC) se fundamenta en los estándares y documentos RFC de la IETF [7–10].

Finalmente, si quisiéramos hacer un pentesting profesional deberíamos seguir los estándares y metodologías descritos en guías como NIST SP 800-115 [11], el estándar PTES [12], la guía de pruebas OWASP WSTG [13] y la matriz de tácticas de reconocimiento de MITRE ATT&CK [14].

Adquisición

```
1 #!/bin/bash
2
3 # Inicializacion de variables.
4 instalar_herramientas=0
5 objetivo=""
6 archivo_salida=""
7 max_subdominios=0
8 omitir_subdominios=0
9 modo_solo_ip=0
10 ip_original=""
11
12 # Guardamos el stdout original (la terminal) en el descriptor 3.
13 # Más adelante, cuando redirijamos todo al archivo, usaremos >&3
14 # para imprimir mensajes de estado que el usuario sí debe ver.
15 exec 3>&1
16
17 # Funcion para desplegar la informacion de ayuda.
18 mostrar_ayuda(){
19     echo "===== " >&3
20     echo "      Script de Pentesting - Práctica 2 de CyS" >&3
21     echo "===== " >&3
22     echo " " >&3
23     echo "Descripción:" >&3
24     echo " Este script realiza una recolección de información sobre" >&3
25     echo " un dominio o dirección IP. Toda la salida se guarda únicamente"
26     echo " en el archivo de reporte; la terminal solo muestra mensajes"
27     echo " de estado del progreso del script." >&3
28     echo " " >&3
29     echo " Si se ingresa una dirección IPv4, el script busca primero" >&3
30     echo " el dominio asociado vía registro PTR, certificado SSL/TLS o"
31     echo " redirección HTTP. Si encuentra un dominio, ejecuta el análisis"
32     echo " completo sobre él; si no, ejecuta un análisis de red, latencia"
33     echo " ruta y puertos directamente sobre la IP (modo solo IP)." >&3
34     echo " " >&3
35     echo " Herramientas utilizadas:" >&3
36     echo "   whois      Información del registrador, contactos y fechas."
37     echo "   ping       Conectividad y latencia." >&3
38     echo "   host/dig   IPs públicas (IPv4/IPv6), segmentos y registros"
39     echo "   reversos." >&3
40     echo "   traceroute Ruta y saltos al objetivo." >&3
41     echo "   dnsrecon   Enumeración completa de registros DNS." >&3
42     echo "   subfinder  Descubrimiento pasivo de subdominios (certificados CT)." >&3
43     echo "   sublist3r  Descubrimiento pasivo de subdominios (motores de búsqueda)." >&3
44     echo "   findomain  Descubrimiento pasivo de subdominios (registros
```

```

SSL)." >&3
44 echo "      dnsmap      Descubrimiento activo de subdominios (fuerza
bruta)." >&3
45 echo "      nmap      Escaneo de puertos abiertos, estados y versión de
servicios." >&3
46 echo "" >&3
47 echo "Uso:" >&3
48 echo " \${0} [banderas] <dominio|ip> [archivo_salida]" >&3
49 echo "" >&3
50 echo "Argumentos:" >&3
51 echo " <dominio|ip>      Dominio (ej. unam.mx) o dirección IPv4. [
REQUERIDO]" >&3
52 echo "                  Si es IP, busca dominio o ejecuta modo solo
IP." >&3
53 echo " [archivo_salida]  Archivo donde se guardará el reporte." >&3
54 echo "                  Por defecto: <dominio|ip>_reporte_pentesting
.txt" >&3
55 echo "" >&3
56 echo "Banderas:" >&3
57 echo " -h, --help          Muestra esta ayuda y finaliza el script
." >&3
58 echo " -i, --install        Verifica e instala las herramientas
necesarias." >&3
59 echo " -s, --skip-subdomains  Omite la búsqueda de subdominios (
recomendado para" >&3
60 echo "                  objetivos masivos como sistemas de
gobierno)." >&3
61 echo " -n <número>          Limita la tabla de subdominios a los
primeros <número>" >&3
62 echo "                  activos. Sin esta bandera se muestra la
tabla completa." >&3
63 echo "                  Si hay más resultados que el límite,
indica cuántos quedan." >&3
64 echo "===== " >&3
65 }
66
67 # Se leen los parametros de entrada.
68 while [[ "$#" -gt 0 ]]; do
69     case $1 in
70         -h | --help)
71             mostrar_ayuda
72             exit 0
73             ;;
74         -i | --install)
75             instalar_herramientas=1
76             shift
77             ;;
78         -s | --skip-subdomains | --no-subdomains)
79             omitir_subdominios=1
80             shift
81             ;;
82         -n)

```

```

83         if [[ "$2" =~ ^[0-9]+$ ]] && [ "$2" -gt 0 ]; then
84             max_subdominios=$2
85             shift 2
86         else
87             echo "Error: -n requiere un número entero positivo. (ej. -
n 50)" >&3
88             exit 1
89         fi
90         ;;
91     -*)
92         echo "Error: Opción desconocida $1" >&3
93         mostrar_ayuda
94         exit 1
95         ;;
96     *)
97         if [ -z "$objetivo" ]; then
98             objetivo=$1
99         elif [ -z "$archivo_salida" ]; then
100             archivo_salida=$1
101         else
102             echo "Argumento extra ignorado -> $1." >&3
103         fi
104         shift
105         ;;
106     esac
107 done
108
109 # Validar que la entrada de objetivo se proporcione.
110 if [ -z "$objetivo" ]; then
111     echo "Error: Parámetro de objetivo faltante." >&3
112     echo "Uso: $0 [-i] <dominio|ip> [archivo_salida]" >&3
113     echo "Ejecuta $0 -h para ayuda." >&3
114     exit 1
115 fi
116
117 #-----
118 # Si el objetivo ingresado es una dirección IPv4, intentamos obtener
119 # el dominio asociado mediante:
120 # 1. Consulta reversa PTR con dig -x.
121 # 2. Inspección del certificado SSL/TLS en el puerto 443.
122 # 3. Cabecera HTTP de redirección (Location).
123 # Si no se encuentra un dominio, se activa el modo solo IP para
124 # realizar las pruebas aplicables a direcciones de red.
125 #-----
126 if [[ "$objetivo" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
127     ip_original="$objetivo"
128     echo "Dirección IPv4 detectada: $ip_original" >&3
129     echo "Buscando dominio asociado mediante registro reverso (PTR)..."
>&3
130
131     ptr_raw=$(dig -x "$ip_original" +short 2>/dev/null | head -n 1)
132     dominio_ptr=$(echo "$ptr_raw" | sed 's/\.$/ /')

```

```

133
134     if [ -n "$dominio_ptr" ]; then
135         echo "Dominio asociado encontrado (PTR): $dominio_ptr" >&3
136         echo "El análisis se ejecutará sobre: $dominio_ptr" >&3
137         objetivo="$dominio_ptr"
138     else
139         echo "No se encontró registro PTR. Consultando certificado SSL/TLS
140         ..." >&3
141         ssl_out=$(echo | timeout 3 openssl s_client -connect "$ip_original
142         :443" 2>/dev/null | openssl x509 -noout -subject -ext subjectAltName
143         2>/dev/null)
144         ssl_dom=$(echo "$ssl_out" | grep -o 'DNS:[a-zA-Z0-9.-]*' | cut -d:
145         -f2 | grep -v '^\' | sort -u | head -n 1)
146         [ -z "$ssl_dom" ] && ssl_dom=$(echo "$ssl_out" | grep -o 'CN\s*=\s
147         *[,]*' | sed -e 's/CN\s*=\s*/' -e 's/\s*/g' | head -n 1)
148
149         if [ -n "$ssl_dom" ]; then
150             echo "Dominio asociado encontrado (Certificado SSL): $ssl_dom"
151             >&3
152             echo "El análisis se ejecutará sobre: $ssl_dom" >&3
153             objetivo="$ssl_dom"
154         else
155             echo "No se encontró certificado SSL. Consultando redirección
156             HTTP..." >&3
157             http_loc=$(timeout 3 curl -s -I -k "https://$ip_original/" 2>/
158             dev/null | grep -i '^location:' | head -n 1 | awk '{print $2}' | sed -E
159             's|https?:/([^:/]+).*\|1|' | tr -d '\r')
160             [ -z "$http_loc" ] && http_loc=$(timeout 3 curl -s -I "http://
161             $ip_original/" 2>/dev/null | grep -i '^location:' | head -n 1 | awk '{
162             print $2}' | sed -E 's|https?:/([^:/]+).*\|1|' | tr -d '\r')
163
164             if [ -n "$http_loc" ]; then
165                 echo "Dominio asociado encontrado (Redirección HTTP):
166                 $http_loc" >&3
167                 echo "El análisis se ejecutará sobre: $http_loc" >&3
168                 objetivo="$http_loc"
169             else
170                 echo "No se encontró ningún dominio asociado para
171                 $ip_original." >&3
172                 echo "Continuando en modo solo IP (análisis de red y
173                 servicios)..." >&3
174                 modo_solo_ip=1
175                 objetivo="$ip_original"
176             fi
177         fi
178     fi
179 fi
180
181 # Valida que se haya dado una ruta de archivo de salida,
182 # en otro caso utiliza el archivo de salida por defecto.
183 if [ -z "$archivo_salida" ]; then
184     archivo_salida="${objetivo}"_reporte_pentesting.txt

```

```
171     echo "No se especificó archivo de salida. Usando archivo por defecto:
172     $archivo_salida" >&3
173 fi
174 #-----
175 # A partir de este punto redirigimos stdout y stderr del proceso
176 # completo al archivo de salida (sobreescribiéndolo si ya existe).
177 # Cualquier herramienta (dnsrecon, traceroute, dig, etc.) escribirá
178 # directo al archivo sin que nada aparezca en la terminal. Los mensajes
179 # de estado al usuario se imprimen con >&3 (terminal original).
180 # Vacíamos el archivo de salida si ya existe para sobreescribirlo,
181 # y luego abrimos el descriptor 1 en modo append (>>) para que todos
182 # los procesos hijos y comandos escriban de forma secuencial al final.
183 > "$archivo_salida"
184 exec 1>>"$archivo_salida" 2>&1
185
186 # Verificar que las herramientas necesarias estén instaladas en el caso
187 # que se haya pasado la bandera -i.
188 if [ $instalar_herramientas -eq 1 ]; then
189     echo "Verificando herramientas..." >&3
190     paquetes=("iputils-ping" "dnsutils" "host" "dig" "traceroute" "whois"
191     "sublist3r" "subfinder" "findomain" "dnsmap" "dnsrecon" "nmap" "
192     etherape" "curl" "openssl")
193     for paquete in "${paquetes[@]}; do
194         comando=$paquete
195         [ "$paquete" == "iputils-ping" ] && comando="ping"
196         [ "$paquete" == "dnsutils" ] && comando="nslookup"
197
198         if ! command -v "$comando" &> /dev/null; then
199             echo "Instalando $paquete..." >&3
200             sudo apt-get update > /dev/null
201             sudo apt-get install -y "$paquete"
202         fi
203     done
204     echo "Comprobación de herramientas completada." >&3
205 fi
206
207 echo "Comienza pruebas de pentesting." >&3
208 echo "Guardando los resultados en $archivo_salida..." >&3
209 echo "Reporte de pentesting para $objetivo"
210 if [ "$modo_solo_ip" -eq 1 ]; then
211     echo "Tipo de objetivo: Dirección IPv4 (Modo solo IP)"
212 fi
213 echo "Fecha de escaneo: $(date)"
214 echo "===== "
215 #-----
216 # Obtenemos información usando WHOIS.
217 # Filtramos con grep -v "^%" para eliminar los comentarios de uso
218 # responsable que whois agrega al final de su salida.
219 #-----
220 echo "Usando WHOIS obtenemos información del objetivo." >> "
```

```

    $archivo_salida"
220 echo "===== "
    >> "$archivo_salida"
221 whois "$objetivo" | grep -v "^%" >> "$archivo_salida"
222 echo "===== "
    >> "$archivo_salida"
223
224 #-----
225 # Comprobamos la conectividad y medimos la latencia usando ping.
226 # Usamos -c 4 para limitar a 4 paquetes y grep -iE para filtrar
227 # únicamente las líneas con el resumen de pérdida de paquetes y RTT.
228 #-----
229 echo "Usando ping comprobamos la conectividad y medimos la latencia." >> "
    $archivo_salida"
230 echo "===== "
    >> "$archivo_salida"
231 echo "" >> "$archivo_salida"
232 ping -c 4 "$objetivo" | grep -iE "^ping|^---|paquetes|rtt" >> "
    $archivo_salida"
233 echo "" >> "$archivo_salida"
234
235 #-----
236 # Buscamos las direcciones IPv4 e IPv6 públicas con host y dig.
237 # En modo solo IP se analiza directamente la segmentación de red
238 # y los registros reversos de la IP objetivo.
239 #-----
240 if [ "$modo_solo_ip" -eq 1 ]; then
241     ipv4="$objetivo"
242     echo "
===== " >> "
    $archivo_salida"
243     echo "Verificando segmentación de red y registros reversos para la IP.
" >> "$archivo_salida"
244     echo "
===== " >> "
    $archivo_salida"
245     echo "" >> "$archivo_salida"
246     echo "-----" >> "$archivo_salida"
247     echo "Analizando la IP $objetivo:" >> "$archivo_salida"
248     echo "Segmentos de red asignados:" >> "$archivo_salida"
249     whois "$objetivo" | grep -iE "CIDR|inetnum|route|NetRange" >> "
    $archivo_salida"
250     echo "" >> "$archivo_salida"
251     echo "Registros reversos:" >> "$archivo_salida"
252     dig -x "$objetivo" +short >> "$archivo_salida"
253     echo "-----" >> "$archivo_salida"
254     echo "===== "
    >> "$archivo_salida"
255 else
256     echo "
===== " >> "
    $archivo_salida"

```

```

257     echo "Usando host obtenemos las direcciones IPv4 e IPv6 del servidor."
258     >> "$archivo_salida"
259     echo "
===== " >> "
$archivo_salida"
259     echo "" >> "$archivo_salida"
260     host "$objetivo" >> "$archivo_salida"
261     echo "" >> "$archivo_salida"
262     echo "Usamos dig para confirmar IPs públicas (+noall +answer)." >> "
$archivo_salida"
263     echo "" >> "$archivo_salida"
264     dig "$objetivo" +noall +answer >> "$archivo_salida"
265     echo "Registros IPv6 (AAAA):" >> "$archivo_salida"
266     dig "$objetivo" AAAA +short >> "$archivo_salida"
267     echo "
===== " >> "
$archivo_salida"

268
269     echo " Verificando segmentación de red y registros reversos por IP."
>> "$archivo_salida"
270     echo "
===== " >> "
$archivo_salida"
271     echo "" >> "$archivo_salida"
272
273     ipv4=$(dig "$objetivo" +short | grep -E '
^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$')
274     if [ -n "$ip_original" ] && ! grep -q "$ip_original" <<< "$ipv4"; then
275     ipv4=$(printf "%s\n%s" "$ipv4" "$ip_original" | sed '/^$/d' | sort
-u)
276     fi
277     if [ -n "$ipv4" ]; then
278         while read -r ip; do
279             echo "-----" >> "
$archivo_salida"
280             echo "Analizando la IP $ip:" >> "$archivo_salida"
281             echo "Segmentos de red asignados:" >> "$archivo_salida"
282             whois "$ip" | grep -iE "CIDR|inetnum|route|NetRange" >> "
$archivo_salida"
283             echo "" >> "$archivo_salida"
284             echo "Registros reversos:" >> "$archivo_salida"
285             dig -x "$ip" +short >> "$archivo_salida"
286             echo "-----" >> "
$archivo_salida"
287             done <<< "$ipv4"
288         else
289             echo "No se pudo obtener ninguna IPv4." >> "$archivo_salida"
290         fi
291
292         echo "===== "
>> "$archivo_salida"
293         echo "Hacemos lo mismo para las IPs IPv6:" >> "$archivo_salida"

```

```

294
295     ipv6=$(dig "$objetivo" AAAA +short | grep -E '^[0-9a-fA-F:]+$')
296     if [ -n "$ipv6" ]; then
297         while read -r ip6; do
298             echo "-----" >> "
$archivo_salida"
299             echo "Analizando la IP $ip6:" >> "$archivo_salida"
300             echo "Segmentos de red asignados:" >> "$archivo_salida"
301             whois "$ip6" | grep -iE "CIDR|inetnum|route|NetRange" >> "
$archivo_salida"
302             echo "" >> "$archivo_salida"
303             echo "Registros reversos:" >> "$archivo_salida"
304             dig -x "$ip6" +short >> "$archivo_salida"
305             echo "-----" >> "
$archivo_salida"
306             done <<< "$ipv6"
307         else
308             echo "No se pudo obtener ninguna IPv6." >> "$archivo_salida"
309         fi
310
311         echo "=====
>> "$archivo_salida"
312 fi
313
314 #-----
315 # Obtenemos la ruta y los saltos para llegar al objetivo con traceroute.
316 # Los saltos intermedios pueden aparecer como * si los routers bloquean
317 # los paquetes ICMP.
318 #-----
319 echo "=====
>> "$archivo_salida"
320 echo "Usando traceroute obtenemos la ruta y los saltos al objetivo." >> "
$archivo_salida"
321 echo "=====
>> "$archivo_salida"
322 echo "" >> "$archivo_salida"
323 traceroute "$objetivo" >> "$archivo_salida"
324 echo "" >> "$archivo_salida"
325 echo "=====
>> "$archivo_salida"
326
327 #-----
328 # Enumeramos los registros DNS con dnsrecon.
329 # Obtiene: topología pública (A, AAAA, NS, DNSSEC), proveedores
330 # externos (MX, SRV) y políticas de autenticación (TXT, SPF, DKIM).
331 # En modo solo IP esta prueba se omite por requerir un nombre de dominio.
332 #-----
333 if [ "$modo_solo_ip" -eq 1 ]; then
334     echo "
=====
$archivo_salida"
335     echo "dnsrecon requiere un nombre de dominio (omitido en modo solo IP)"

```

```

336     ." >> "$archivo_salida"
337     echo "
338     ===== " >> "
339     $archivo_salida"
340 else
341     echo "
342     ===== " >> "
343     $archivo_salida"
344     echo "Usando dnsrecon enumeramos los registros DNS del dominio." >> "
345     $archivo_salida"
346     echo "
347     ===== " >> "
348     $archivo_salida"
349     echo "" >> "$archivo_salida"
350     dnsrecon -d "$objetivo" &>> "$archivo_salida"
351     echo "" >> "$archivo_salida"
352     echo "
353     ===== " >> "
354     $archivo_salida"
355 fi
356 # -----
357 # Descubrimos subdominios con cuatro herramientas complementarias:
358 # - subfinder (pasiva): consulta certificados CT y APIs públicas.
359 # - sublist3r (pasiva): consulta motores de búsqueda y APIs.
360 # - findomain (pasiva): especializada en registros de certificados.
361 # - dnsmap (activa): fuerza bruta con diccionario interno.
362 # Cada herramienta guarda su salida en un archivo temporal; los
363 # resultados se unifican con sort -u y se presenta solo la tabla final.
364 # En modo solo IP esta prueba se omite por requerir un nombre de dominio.
365 # Con la bandera -s (--skip-subdomains), se omite para evitar demoras
366 # excesivas en infraestructuras gubernamentales o de gran escala.
367 # -----
368 if [ "$modo_solo_ip" -eq 1 ]; then
369     echo "
370     ===== " >> "
371     $archivo_salida"
372     echo "Descubrimiento de subdominios omitido (requiere nombre de
373     dominio)." >> "$archivo_salida"
374     echo "
375     ===== " >> "
376     $archivo_salida"
377 elif [ "$omitir_subdominios" -eq 1 ]; then
378     echo "
379     ===== " >> "
380     $archivo_salida"
381     echo "Descubrimiento de subdominios omitido por bandera (-s / --skip-
382     subdomains)." >> "$archivo_salida"
383     echo "
384     ===== " >> "
385     $archivo_salida"
386     echo "Fase de descubrimiento de subdominios omitida por el usuario (-s

```

```
) ." >&3
368 else
369     echo "
===== " >> "
$archivo_salida"
370     echo "Descubrimiento de subdominios con múltiples herramientas." >> "
$archivo_salida"
371     echo "
===== " >> "
$archivo_salida"
372     echo "" >> "$archivo_salida"
373
374     tmp_subfinder="/tmp/${objetivo}_subfinder.txt"
375     tmp_sublist3r="/tmp/${objetivo}_sublist3r.txt"
376     tmp_findomain="/tmp/${objetivo}_findomain.txt"
377     tmp_dnsmap="/tmp/${objetivo}_dnsmap.txt"
378     tmp_todos="/tmp/${objetivo}_todos.txt"
379
380     touch "$tmp_subfinder" "$tmp_sublist3r" "$tmp_findomain" "$tmp_dnsmap"
381
382     if command -v subfinder &> /dev/null; then
383         subfinder -d "$objetivo" -silent -o "$tmp_subfinder" > /dev/null
2>&1
384         count_subfinder=$(wc -l < "$tmp_subfinder")
385     else
386         count_subfinder=0
387     fi
388
389     if command -v sublist3r &> /dev/null; then
390         sublist3r -d "$objetivo" -o "$tmp_sublist3r" > /dev/null 2>&1
391         count_sublist3r=$(wc -l < "$tmp_sublist3r")
392     else
393         count_sublist3r=0
394     fi
395
396     if command -v findomain &> /dev/null; then
397         findomain -t "$objetivo" -u "$tmp_findomain" > /dev/null 2>&1
398         count_findomain=$(wc -l < "$tmp_findomain")
399     else
400         count_findomain=0
401     fi
402
403     if command -v dnsmap &> /dev/null; then
404         dnsmap "$objetivo" -r "$tmp_dnsmap" > /dev/null 2>&1
405         count_dnsmap=$(wc -l < "$tmp_dnsmap")
406     else
407         count_dnsmap=0
408     fi
409
410     cat "$tmp_subfinder" "$tmp_sublist3r" "$tmp_findomain" "$tmp_dnsmap"
2>/dev/null \
411     | grep -v '^$' | sort -u > "$tmp_todos"
```

```

412     total=$(wc -l < "$tmp_todos")
413
414
415     echo "
===== " >> "
$archivo_salida"
416     echo "RESUMEN POR HERRAMIENTA:" >> "$archivo_salida"
417     echo "    subfinder    : $count_subfinder subdominios" >> "$archivo_salida"
418     echo "    sublist3r    : $count_sublist3r subdominios" >> "$archivo_salida"
419     echo "    findomain    : $count_findomain subdominios" >> "$archivo_salida"
420     echo "    dnsmap       : $count_dnsmap subdominios" >> "$archivo_salida"
421     echo "
----- " >> "
$archivo_salida"
422     echo "    UNIÓN TOTAL (sin duplicados): $total subdominios únicos" >> "
$archivo_salida"
423     echo "
===== " >> "
$archivo_salida"
424     echo "" >> "$archivo_salida"
425
426     echo "
===== " >> "
$archivo_salida"
427     echo "UNIÓN DE SUBDOMINIOS CON SUS IPs RESUELTAS:" >> "$archivo_salida"
428     if [ "$max_subdominios" -gt 0 ]; then
429         echo "(Mostrando los primeros $max_subdominios subdominios activos
)" >> "$archivo_salida"
430     else
431         echo "(Subdominios con registro DNS activo)" >> "$archivo_salida"
432     fi
433     echo "
===== " >> "
$archivo_salida"
434     printf "%-55s %s\n" "SUBDOMINIO" "IP(s)" >> "$archivo_salida"
435     echo "
----- " >> "
$archivo_salida"
436
437     activos=0
438     restantes=0
439
440     if [ -s "$tmp_todos" ]; then
441         while read -r sub; do
442             ips=$(dig "$sub" +short 2>/dev/null \
443                 | grep -E '^([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$' \
444                 | tr '\n' ',' | sed 's/,,$//')
445             if [ -n "$ips" ]; then

```

```

446         activos=$((activos + 1))
447         if [ "$max_subdominios" -gt 0 ] && [ "$activos" -gt "
$max_subdominios" ]; then
448             restantes=$((restantes + 1))
449         else
450             printf "%-55s %s\n" "$sub" "$ips" >> "$archivo_salida"
451         fi
452     fi
453     done < "$tmp_todos"
454 fi
455
456     echo "
-----" >> "
$archivo_salida"
457     echo "Total de subdominios activos (con IP resuelta): $activos" >> "
$archivo_salida"
458     if [ "$restantes" -gt 0 ]; then
459         echo "... y $restantes subdominios activos más no mostrados." >> "
$archivo_salida"
460     fi
461     echo "
===== " >> "
$archivo_salida"
462
463     rm -f "$tmp_subfinder" "$tmp_sublist3r" "$tmp_findomain" "$tmp_dnsmap"
"$tmp_todos"
464 fi
465
466 #-----
467 # Escaneamos puertos, estados y servicios con nmap.
468 # Usamos -sV para detectar la versión de cada servicio, --open para
469 # mostrar solo los puertos abiertos y -T4 para acelerar el escaneo.
470 # Reutilizamos la lista de IPs obtenida previamente para realizar
471 # la ejecución iterada de nmap sobre cada dirección disponible.
472 #-----
473 echo "===== "
>> "$archivo_salida"
474 echo "Usando nmap obtenemos los puertos abiertos, estados y servicios." >>
"$archivo_salida"
475 echo "===== "
>> "$archivo_salida"
476 echo "" >> "$archivo_salida"
477
478 if command -v nmap &> /dev/null; then
479     if [ -n "$ipv4" ]; then
480         while read -r ip; do
481             echo "-----" >> "
$archivo_salida"
482             echo "Escaneando la IP $ip con nmap:" >> "$archivo_salida"
483             nmap -sV --open -T4 "$ip" >> "$archivo_salida"
484             echo "-----" >> "
$archivo_salida"

```

```

485         echo "" >> "$archivo_salida"
486         done <<< "$ip4"
487     else
488         echo "--- Escaneando el objetivo: $objetivo ---" >> "
$archivo_salida"
489         nmap -sV --open -T4 "$objetivo" >> "$archivo_salida"
490     fi
491 else
492     echo "nmap no está instalado. Omitiendo escaneo de puertos." >> "
$archivo_salida"
493     echo "Instálalo con: sudo apt-get install nmap (o usa la bandera -i)"
>> "$archivo_salida"
494 fi
495
496 echo "" >> "$archivo_salida"
497 echo "=====
>> "$archivo_salida"
498 echo "" >> "$archivo_salida"
499 echo "=====
>> "$archivo_salida"
500 echo "Fin del reporte de pentesting para $objetivo." >> "$archivo_salida"
501 echo "=====
>> "$archivo_salida"
502
503 echo ""
504 echo "Reporte finalizado. Resultados guardados en: $archivo_salida" >&3

```

Para automatizar la obtención de información, desarrollamos el script `pentesting.sh`. Este script recopila los datos públicos y técnicos de un dominio o dirección IP, guardando el resultado en un archivo de reporte (por defecto, `<objetivo>_reporte_pentesting.txt`).

El script incluye banderas para su control:

- **-h, -help:** Muestra las opciones y modo de uso.
- **-i, -install:** Verifica e instala las herramientas necesarias si faltan en el sistema. Funciona únicamente en sistemas que usen `apt-get` como lo es Kali Linux.
- **-s, -skip-subdomains:** Omite por completo la fase de búsqueda y resolución de subdominios. Se añadió esta opción al tratar con el dominio `gob.mx`, ya que la ejecución se prolongaba excesivamente debido a la gran cantidad de subdominios asociados a la infraestructura gubernamental. Este aspecto se detalla con mayor profundidad en la sección de análisis.
- **-n <número>:** Limita la tabla de subdominios a los primeros *n* activos para evitar reportes excesivamente largos.

Si se ingresa una dirección IPv4, el script busca su dominio mediante consulta reversa (PTR), certificado SSL/TLS o redirección HTTP. Si lo encuentra, analiza el dominio completo; si no, entra en *modo solo IP* y ejecuta únicamente las pruebas aplicables a redes y puertos sin detenerse.

El script utiliza las siguientes herramientas para obtener información:

1. **WHOIS:** Obtiene datos del registro, fechas de expiración, contactos y servidores de nombres.
2. **Ping:** Mide la conectividad, latencia y pérdida de paquetes mediante 4 solicitudes ICMP.
3. **Host y Dig:** Identifica las direcciones IP (IPv4 e IPv6), así como los segmentos de red asignados (bloques CIDR e *inetnum*) y registros reversos de cada una.
4. **Traceroute:** Muestra la ruta de saltos y enrutadores hacia el objetivo.
5. **Dnsrecon:** Enumera los registros DNS principales (SOA, MX, TXT) y políticas de correo como SPF y DMARC.
6. **Subdominios:** Combina las herramientas `subfinder`, `sublist3r`, `findomain` y `dnsmap` y elimina duplicados con `sort -u`. Finalmente, genera una tabla con los subdominios que tienen IP activa (a menos que se active la bandera `-s` para omitir esta fase).
7. **Nmap:** Escanea los puertos abiertos (`-open`) y detecta versiones de servicios (`-sV`), iterando sobre todas las IPs disponibles del objetivo.

Procesamiento

Filtramos la información cruda de las herramientas limpiándola del ruido y comentarios siguiendo las siguientes estrategias para facilitar la lectura e interpretación del resultado:

- **Limpieza de texto:** En las consultas de WHOIS filtramos mediante expresiones regulares los avisos de privacidad y licencias para quedarnos solo con los datos de contacto, fechas y servidores de nombres. De igual forma, en las IPs extraemos exclusivamente los bloques CIDR y rangos de red asignados. Además, en las pruebas de conectividad con `ping` filtramos el texto completo para aislar directamente el porcentaje de paquetes perdidos y las estadísticas de latencia (mínima, media y máxima).
- **Normalización y verificación de subdominios:** Al combinar herramientas se generan algunos resultados repetidos. El script los unifica, elimina duplicados con `sort -u`, descarta aquellos que ya no resuelven a una IP activa mediante `dig` y les da formato de tabla. Además, con la bandera `-n` limitamos la cantidad de filas mostradas para que el reporte no se vuelva excesivamente largo, o podemos prescindir completamente de este paso mediante `-s` en dominios muy extensos.
- **Aislamiento de puertos útiles:** En Nmap descartamos los puertos cerrados o filtrados (`-open`), agrupando los resultados por IP con sus versiones de servicio identificadas.

Análisis

A partir de los reportes generados por nuestro script `pentesting.sh`, podemos realizar un análisis de amenazas para los cinco dominios analizados: `unam.mx`, `pemex.com`, `gob.mx`, `ipn.mx` y nuestro objetivo de pruebas elegido, `hackthissite.org`. Todo este análisis va apegado conforme a las bases de datos oficiales de vulnerabilidades como la NVD [1], el catálogo CVE [2] y Exploit-DB [3]:

1. Dominio: `unam.mx`

- **Infraestructura y balanceo de carga:** A diferencia de otros objetivos, los sistemas centrales de la UNAM resuelven a cuatro direcciones IPv4 activas (`132.248.166.17`, `.18`, `.19` y `.20`). Esto probablemente esté hecho para distribuir y balancear la alta carga de peticiones concurrentes hacia el portal principal.
- **Servicios expuestos y versiones (Nmap):** El escaneo de puertos detectó servicios web diferenciados según el puerto:

```
1 PORT      STATE SERVICE  VERSION
2 80/tcp    open  http      Apache httpd 2.4.51 ((Unix))
3 443/tcp   open  ssl/http  nginx 1.29.5
```

Podemos ver el servidor web Apache httpd 2.4.51 en el puerto 80 y Nginx 1.29.5 en el puerto 443 operando como proxy inverso y terminador SSL. Al contrastar la versión de Apache en la *National Vulnerability Database* (NVD) [1], identificamos dos vulnerabilidades importantes:

1. **CVE-2021-44790 (CVSS 9.8 Crítico)** [15]: Desbordamiento de búfer (*buffer overflow*) en el módulo `mod_lua` que podría permitir una denegación de servicio o la ejecución remota de código.
 2. **CVE-2021-44224 (CVSS 7.5 Alto)** [16]: Desreferencia a un puntero nulo en `mod_proxy` que puede ser provocada por peticiones malformadas, forzando la caída del servicio.
- **Exposición de información en WHOIS e Ingeniería Social:** Al utilizar WHOIS vemos que deja expuesta la información de los contactos administrativo y técnico (nombres completos del personal de la DG TIC). Si bien es información pública en registros institucionales, esta información podría ser aprovechada por un atacante para planear ataques de ingeniería social, intentando suplantar a las autoridades ante el personal técnico para obtener accesos indebidos.
 - **Autenticación de correo y vulnerabilidad a Spoofing (DMARC):** De `dnsrecon` se extrajo la siguiente política de correo:

```
1 TXT _dmarc.unam.mx v=DMARC1; p=none; rua=mailto:rua_dmarc@unam.mx; ruf
  =mailto:reporterua_dmarc@unam.mx; fo=0:1;
```

De acuerdo con el RFC 7489 [10], el parámetro `p=none` indica que los proveedores receptores de correo deben dejar pasar a la bandeja de entrada normal los mensajes que no superen las pruebas de autenticación SPF o DKIM, limitándose únicamente a mandar un reporte. Esto hace que el dominio `unam.mx` sea vulnerable a ataques de suplantación de correo (*email spoofing*), facilitando fraudes y engaños hacia sus usuarios.

- **Superficie de ataque en subdominios y Shadow IT:** Se descubrieron 4664 subdominios únicos. Si se hace un análisis de los nombres y las IP se pueden encontrar sitios web creados para eventos o congresos pasados como, por ejemplo, `36congresociss2019.unam.mx` que continúan en línea sin mantenimiento evidente, dándonos un vector de ataque si es que estos subdominios no son mantenidos apropiadamente o eliminados.
- **Canales de contacto y reporte para divulgación responsable:** Para notificar formalmente las vulnerabilidades identificadas, la institución dispone de los siguientes canales de coordinación:
 - **UNAM-CERT (Equipo de Respuesta a Incidentes de Seguridad en Cómputo):** Adscrito al Departamento de Seguridad en Cómputo de la Dirección General de Cómputo y de Tecnologías de Información y Comunicación (DGTIC). Punto de contacto principal para reportes de incidentes y brechas: correo `cert@unam.mx` y portal oficial de reporte en línea en `https://www.seguridad.unam.mx/`.
 - **Contactos técnicos y administrativos (WHOIS):** Personal responsable de la operación de red registrado ante NIC México en la DGTIC (Circuito Exteriores/n, Ciudad Universitaria, Coyoacán, CDMX).
 - **Buzones de telemetría y reportes forenses DMARC:** Cuentas receptoras configuradas en los registros DNS para anomalías de correo: `rua_dmarc@unam.mx` y `reporterua_dmarc@unam.mx`.

2. Dominio: `pemex.com`

- **Infraestructura y direccionamiento:** Resuelve a una dirección IPv4 pública (`200.23.91.20`) administrada directamente por Petróleos Mexicanos bajo el segmento `200.23.91.0/24`.
- **Protección perimetral y detección de WAF (F5 BIG-IP):** Aunque Nmap reporta los puertos `80/tcp` y `443/tcp` abiertos con servicios genéricos, la captura de cabeceras HTTP durante el escaneo arrojó firmas características de cookies de sesión:

```

1 PORT      STATE SERVICE  VERSION
2 80/tcp    open  http      httpd/2.4.6
3 443/tcp    open  ssl/https
4 Set-Cookie: PMX_Cookie=!FFsUDrWbLRwc...; path=/; Httponly
5 Set-Cookie: f5avraaaaaaaaaaaaaaaaa_session=DJEJXHDC...; HttpOnly;
6 Set-Cookie: TS0136b7a5=01d42166bbcc21e5...; Path=/;

```

Las cookies con prefijos `f5avr...`, `TS...` y el nombre de la organización demuestran la presencia de un balanceador y cortafuegos de aplicaciones web (*WAF*) **F5 Networks BIG-IP**. Si bien este equipo filtra ataques comunes, la infraestructura F5 ha presentado vulnerabilidades críticas documentadas en NVD (como **CVE-2022-1388** [17] y **CVE-2023-46747** [18]) que permiten la omisión de autenticación en consolas de gestión y la ejecución remota de comandos si las interfaces administrativas quedan expuestas.

- **Autenticación de correo (DMARC):**

```
1 TXT _dmarc.pemex.com v=DMARC1;p=quarantine;adkim=s;aspf=s;fo=1;ri=3600;pct=100;rua=mailto:reportesdmarc@pemex.com;ruf=mailto:reportesdmarc@pemex.com
```

A diferencia de la UNAM, Pemex implementa una política robusta con `p=quarantine` al 100 % de los correos (`pct=100`) y alineación estricta tanto en DKIM (`adkim=s`) como en SPF (`aspf=s`). Esto ordena que cualquier correo que falsifique el dominio sea automáticamente canalizado a la carpeta de correo no deseado, reduciendo drásticamente el riesgo de *spoofing*.

- **Exposición de entornos de desarrollo y APIs en subdominios:** El script identificó 284 subdominios únicos (125 activos). Entre ellos destacan entornos de desarrollo y control de calidad expuestos públicamente hacia Internet:
 - `api-cloud-des.pemex.com` y `ase3-digital-des.pemex.com` (Entornos de Desarrollo).
 - `api-cloud-qa.pemex.com` y `ase3-digital-qa.pemex.com` (Entornos de Pruebas / QA).
 - APIs comerciales directas (`api.pemex.com`, `apicomercializadores.pemex.com`) y servicios filiales en otros rangos de IP (`api.gasbienestar.pemex.com` en 187.218.217.12).

Estos servidores de prueba representan un riesgo considerable, ya que suelen contar con versiones preliminares, políticas de contraseñas débiles o mensajes de depuración activos.

- **Canales de contacto y reporte para divulgación responsable:** Para reportar las vulnerabilidades detectadas en la información del script o en una auditoría de seguridad más profunda, se cuenta con los siguientes canales:
 - **CSIRT Pemex / Gerencia de Tecnologías de Información y Ciberseguridad:** Es el equipo operativo de ciberseguridad industrial y corporativa de Petróleos Mexicanos. Su canal de contacto técnico es `seguridad.informatica@pemex.com`.
 - **Buzón directo de seguridad DMARC:** Correo institucional para incidentes y reportes agregados/forenses de tráfico de correo: `reportesdmarc@pemex.com` (extraído de los parámetros `rua` y `ruf` de su registro DMARC).

- **Contacto corporativo (WHOIS):** Oficinas Centrales de Petróleos Mexicanos, Av. Marina Nacional 329, Col. Petróleos Mexicanos, Alcaldía Miguel Hidalgo, C.P. 11311, Ciudad de México.

3. Dominio: `gob.mx`

- **Infraestructura de red y direccionamiento troncal:** El portal central resuelve a la dirección IPv4 pública 207.249.122.223, perteneciente al bloque institucional 207.248.0.0/15 (`inetnum: 207.249.96.0/19`) registrado ante LACNIC [4] y operado en la infraestructura troncal de Bestel Clientes (`bestelclientes.com.mx`) y Cogent Communications (`atlas.cogentco.com`). En las pruebas de conectividad obtuvimos una latencia media de 37.97 ms sin pérdida de paquetes (0%). En el rastreo de ruta (`traceroute`), a partir del salto 16 los enrutadores descartan los paquetes ICMP (* * *), evidenciando filtrado perimetral estricto.
- **Servicios expuestos y buenas prácticas de ocultamiento (Nmap):** El escaneo de puertos sobre la IP 207.249.122.223 arrojó únicamente los dos servicios web estándar:

```
1 PORT      STATE SERVICE  VERSION
2 80/tcp    open  http     nginx
3 443/tcp   open  ssl/http nginx
```

A diferencia de otros objetivos que exponen la versión exacta del software (como Apache 2.4.51 en la UNAM o HAProxy en HackThisSite), la administración de `gob.mx` suprime el número de versión de Nginx en las cabeceras HTTP, lo que evita que atacantes identifiquen directamente vulnerabilidades específicas asociadas a versiones desactualizadas.

- **Gestión de dominio de segundo nivel y servidores raíz (WHOIS y DNS):** La consulta WHOIS muestra que la titularidad y los contactos administrativos y técnicos pertenecen directamente a *Network Information Center, S.A. de C.V.* (NIC México, Monterrey, N.L.). Esto se debe a que `gob.mx` opera formalmente como un dominio de segundo nivel dentro del ccTLD `.mx`, reservado para el Estado mexicano. Los servidores de nombres autoritativos corresponden a la infraestructura raíz nacional (`m.mx-ns.mx`, `e.mx-ns.mx`, `x.mx-ns.mx`, etc.), los cuales exponen la versión de BIND en sus respuestas DNS.
- **Políticas de autenticación de correo y ausencia de DMARC:** En la enumeración con `dnsrecon` se extrajo el siguiente registro de correo:

```
1 TXT gob.mx v=spf1 -all
```

El parámetro `-all` implementa un rechazo estricto (*Hard Fail*) mediante SPF [9], indicando que el dominio raíz desnudo `gob.mx` no autoriza a ningún servidor en el mundo para emitir correos en su nombre. Sin embargo, al consultar los registros DMARC (`_dmarc.gob.mx`) se constata que **no existe una política DMARC publicada**. De acuerdo con el RFC 7489 [10], la ausencia de DMARC deja la decisión de rechazo al arbitrio exclusivo del filtro antispam del destinatario y priva a la organización de los

mecanismos de telemetría y reportes forenses agregados (rua) para detectar campañas de suplantación que utilicen su nombre.

- **Proliferación masiva de subdominios, saturación del script y riesgos críticos de seguridad:**
 - **Origen de la gran cantidad de subdominios:** A diferencia de los demás dominios de la práctica (que corresponden a una universidad o a una empresa específica), **gob.mx** funciona como la sombrilla digital de toda la Administración Pública Federal de México. Alberga todas las secretarías de Estado (Gobernación, Hacienda, Relaciones Exteriores, Bienestar, Salud, Educación, etc.), cientos de órganos desconcentrados, institutos y comisiones reguladoras (SAT, IMSS, ISSSTE, CONAH-CYT, INEGI, PROFECO, CFE, etc.), poderes de la unión, portales de trámites ciudadanos masivos (CURP, cédulas profesionales, pasaportes, becas), así como campañas temporales, foros y micrositios creados a lo largo de sucesivos sexenios. En los registros públicos de transparencia de certificados (Certificate Transparency) y zonas DNS, **gob.mx** acumula decenas de miles de subdominios.
 - **Impacto en la ejecución del script y motivación de la bandera -s:** Al ejecutar el script **pentesting.sh** completo sobre **gob.mx**, las herramientas pasivas (**subfinder**, **sublist3r**, **findomain**) recopilan decenas de miles de nombres. Posteriormente, al iterar con **dig** para verificar cuáles subdominios tienen IP activa, el proceso toma varias horas. Este comportamiento demostró en la práctica la necesidad de implementar una bandera como **-s**, **-skip-subdomains**, permitiendo omitir esta fase en infraestructuras gubernamentales masivas y enfocar el análisis en los servicios principales. Esto permite notar que la seguridad de un sistema también se puede ver tanto afectada como beneficiada por la magnitud del mismo: dado que los subdominios son tantos, la complejidad de analizar cada uno de ellos en búsqueda de un vector de ataque es tan alta que puede dificultar un escaneo superficial; sin embargo, este aspecto suele jugar en contra, pues una superficie tan dispersa incrementa la probabilidad de puntos ciegos.
 - **Riesgos de seguridad de una superficie de subdominios hipermasiva:**
 1. **Secuestro de subdominios (*Subdomain Takeover*):** Con miles de subdominios creados a lo largo de los años para campañas o micrositios alojados en proveedores en la nube (AWS S3, Microsoft Azure, GitHub Pages, Cloudflare, etc.), es frecuente que un servicio en la nube se cancele pero el registro DNS **CNAME** permanezca apuntando al identificador antiguo. Un atacante puede registrar dicho recurso en el proveedor de nube y tomar el control total del subdominio legítimo bajo **.gob.mx** (vulnerabilidad tipificada en OWASP WSTG-CONF-10 [13] y clasificada como CWE-284 [19]), pudiendo emitir certificados SSL válidos, alojar portales falsos de recaudación o distribuir malware con la credibilidad total del gobierno federal.
 2. **Desgobierno tecnológico y Shadow IT:** Cada dependencia, secretaría o proveedor externo subcontratado administra sus propios servidores con niveles

de seguridad totalmente dispares. Mientras el portal central `gob.mx` mantiene buenas defensas perimetrales y versiones ocultas, subdominios secundarios desatendidos pueden estar corriendo manejadores de contenido vulnerables (WordPress, Joomla, Drupal) o versiones antiguas de PHP y bases de datos sin actualizar durante años.

3. **Riesgo de fuga y cruce de cookies de sesión (*Cookie Scope*):** Si alguna aplicación gubernamental en un subdominio o en el dominio raíz define cookies con alcance amplio (`Domain=.gob.mx`), un fallo de Cross-Site Scripting (XSS) en cualquier subdominio descuidado de un municipio o secretaría menor podría permitir a un atacante capturar tokens de sesión o inyectar cookies fraudulentas hacia aplicaciones críticas del gobierno federal.
 4. **Superficie de ataque expansiva y sistemas heredados (*Legacy*):** La enorme cantidad de puntos de entrada dificulta el inventario y monitoreo continuo por parte de los centros de operaciones de seguridad (SOC/CERT gubernamentales), dejando puntos ciegos que facilitan el acceso inicial de adversarios.
- **Canales de contacto y reporte para divulgación responsable:** Al tratarse de la plataforma central del Estado mexicano y con miras a mitigar el riesgo de secuestro de subdominios (*subdomain takeover*) y la ausencia de políticas DMARC, los reportes deben dirigirse a las instancias de coordinación de seguridad digital gubernamental:
 - **CERT-MX (Guardia Nacional):** Centro Nacional de Respuesta a Incidentes Cibernéticos de la Dirección General Científica de la Guardia Nacional (entidad técnica responsable a nivel federal de coordinar incidentes de ciberseguridad en dependencias gubernamentales e infraestructura crítica). Correo de notificación: `cert-mx@gn.gob.mx`, atención ciudadana: `contacto.ciudadano@gn.gob.mx`, teléfono nacional: 088 y sitio web: <https://cert.mx/>.
 - **Mesa técnica de atención y operación de gob.mx:** Coordinación de Estrategia Digital Nacional (CEDN, Oficina de la Presidencia) y Unidad de Gobierno Digital (Secretaría Anticorrupción y Buen Gobierno / Función Pública). Correo de atención y soporte técnico: `gobmx@funcionpublica.gob.mx`.
 - **Contacto de administración de dominio (WHOIS):** Network Information Center México (NIC México, Monterrey, N.L.) vía `info@nic.mx`, responsable de la delegación del dominio de segundo nivel `.gob.mx`.

4. Dominio: `ipn.mx`

- **Infraestructura alojada en la nube (Microsoft Azure):** El dominio principal resuelve a la dirección IP `20.64.80.120`, la cual pertenece a la red de centros de datos de Microsoft Corporation (`20.64.0.0/10`).
- **Pasarela de aplicaciones web y servicios (Nmap):**

1	PORT	STATE	SERVICE	VERSION
2	80/tcp	open	http	Microsoft Azure Application Gateway v2
3	443/tcp	open	ssl/http	Microsoft Azure Application Gateway v2

El servicio expuesto es *Microsoft Azure Application Gateway v2*, actuando como balanceador de carga. Este esquema oculta y aísla los servidores de backend institucionales. El vector de auditoría principal en este tipo de servicio no se enfoca en fallos binarios del software servidor, sino en técnicas de evasión de reglas WAF o en la búsqueda de la IP pública real del backend en caso de que carezca de restricciones de firewall en la nube.

- **Autenticación de correo de máxima seguridad (DMARC):**

```
1 TXT _dmarc.ipn.mx v=DMARC1; p=reject; pct=100; rua=mailto:correo@ipn.mx; ruf=mailto:dmarc_forensic@ipn.mx; fo=1
```

El IPN tiene implementada la política más estricta posible: **p=reject al 100 %**. Todo correo electrónico no autenticado es rechazado y descartado de inmediato por los servidores receptores en el mundo, impidiendo completamente la suplantación de identidad mediante correos falsos con dominio @ipn.mx.

- **Deducción de direccionamiento interno mediante subdominios:** Se encontraron más de 2,200 subdominios activos. Al analizarlos, se descubrió un esquema de nomenclatura secuencial predecible:

- 21.servidor.dev.ipn.mx → 148.204.111.21
- 22.servidor.upev.ipn.mx → 148.204.111.22
- 23.servidor.dev.ipn.mx → 148.204.111.23
- 24.servidor.dev.ipn.mx → 148.204.111.24

Este patrón revela que el nombre del subdominio coincide con el último octeto de la dirección IP asignada dentro del segmento institucional 148.204.111.0/24. Esto permite deducir la topología de servidores de desarrollo (**dev**) y de la Unidad Politécnica para la Educación Virtual (**upev**), facilitando la enumeración de equipos sin necesidad de escaneos masivos.

- **Canales de contacto y reporte para divulgación responsable:** Para la notificación y reporte de estas vulnerabilidades, se cuenta con los siguientes canales institucionales:

- **CSIRT-IPN / Coordinación del Sistema Institucional de Información y las Comunicaciones (CSIIC):** Dependencia central a cargo de la infraestructura tecnológica, la Red de Telecomunicaciones (PoliRed) y la seguridad de la información institucional. Canales de soporte y atención de incidentes: **ayudapolired@ipn.mx** y **correo@ipn.mx**.

- **Buzón forense DMARC:** Correo institucional designado para recepción de reportes de fallas forenses de autenticación: `dmarc_forensic@ipn.mx` (obtenido del campo `ruf` de su registro DMARC).
- **Ubicación y contacto físico institucional:** Unidad Profesional Adolfo López Mateos, Av. Juan de Dios Bátiz s/n, Col. Nueva Industrial Vallejo, Zacatenco, Alcaldía Gustavo A. Madero, C.P. 07738, Ciudad de México.

5. Dominio a elección: `hackthissite.org`

- **Infraestructura distribuida:** El dominio resuelve a cinco direcciones IPv4 simultáneas (137.74.187.100 a .104) alojadas en la infraestructura del proveedor europeo OVH SAS (Roubaix, Francia), distribuyendo el tráfico de los retos y foros.

- **Servicios expuestos y versiones (Nmap):**

	PORT	STATE	SERVICE	VERSION
2	80/tcp	open	http-proxy	HAProxy http proxy 1.3.1 - 1.9.0
3	443/tcp	open	ssl/http-proxy	HAProxy http proxy 1.3.1 - 1.9.0

El escaneo reporta el balanceador y proxy inverso **HAProxy** en versiones intermedias. Al contrastar este rango en la NVD se encuentran vulnerabilidades históricas importantes:

1. **CVE-2019-18277 (CVSS 7.5 Alto)** [20]: Vulnerabilidad a *HTTP Request Smuggling* generada por discrepancias en el manejo conjunto de las cabeceras **Transfer-Encoding** y **Content-Length**.
2. **CVE-2020-11100 (CVSS 8.8 Alto)** [21]: Desbordamiento de búfer en el procesamiento de tramas HPACK en conexiones HTTP/2.

- **Autenticación de correo permisiva (DMARC):**

```
1 TXT _dmarc.hackthissite.org v=DMARC1;p=quarantine;sp=quarantine;fo=0:1:d:s;aspf=r;adkim=r;ri=86400;pct=25;rua=mailto:8rbjyycl@ag.dmarcian.eu;ruf=mailto:8rbjyycl@fr.dmarcian.eu;
```

Aunque el dominio declara la política `p=quarantine`, incluye el modificador `pct=25`. Esto significa que **solo el 25 % de los correos suplantados son enviados a cuarentena**, permitiendo que el 75 % restante ingrese libremente a la bandeja de entrada, lo cual representa una configuración permisiva y riesgosa frente a campañas de suplantación, aunque al tratarse de un dominio que no pertenece a una institución pública puede ser que sea menos vulnerable a ataques de *email spoofing*.

- **Mapeo de subdominios:** Se recopilaron 70 subdominios únicos (31 activos), correspondientes a módulos de prácticas, retos de seguridad específicos, servidores IRC de la comunidad y foros de discusión.

- **Canales de contacto y reporte para divulgación responsable:** Por tratarse de una comunidad pública y abierta orientada a la educación en seguridad informática y piratería ética, disponen de diversos mecanismos comunitarios y directos para interactuar con su equipo de administración:
 - **Equipo de desarrollo y administración central:** Buzón de contacto con los administradores del proyecto: `admin@hackthissite.org`.
 - **Formulario oficial de contacto web:** Portal directo para reportes técnicos y contacto con el staff: `https://www.hackthissite.org/pages/contact/contact.php`.
 - **Canal IRC en tiempo real:** Red `irc.hackthissite.org`, canal `#hackthissite` (permite establecer contacto inmediato con desarrolladores, operadores de red y moderadores de la plataforma).
 - **Buzón de telemetría DMARC:** Cuentas de reporte del servicio externo de monitoreo `dmarcian`: `8rbjyycl@ag.dmarcian.eu` y `8rbjyycl@fr.dmarcian.eu`.

Propuestas

Respuesta:

Teniendo en cuenta las vulnerabilidades, configuraciones débiles y datos de inteligencia obtenidos, formulamos tanto las propuestas de vectores de ataque desde la perspectiva de un atacante como las propuestas de mitigación y defensa.

1. Propuestas de vectores de ataque (Perspectiva Ofensiva)

1. Campaña de suplantación institucional (*Email Spoofing* y *Phishing*):

- **Objetivos:** `unam.mx`, `hackthissite.org` y `gob.mx`.
- **Incumplimiento técnico:** Violación a las directrices de aplicación de políticas del estándar **RFC 7489 (DMARC)** [10]. En `unam.mx` se incumple el objetivo de protección al mantener `p=none` (modo solo monitoreo), en `hackthissite.org` al usar `pct=25` (dejando el 75 % del tráfico sin protección), y en `gob.mx` al omitir por completo la publicación de un registro DMARC, desaprovechando la telemetría forense y el forzado de rechazo en servidores receptores.
- **Mecánica del ataque:** Un atacante puede montar un servidor propio y emitir correos fraudulentos suplantando direcciones legítimas como `rectoria@unam.mx`, `contacto@gob.mx` o `escolar@unam.mx`. Al recibir los correos, los proveedores destinatarios (Gmail, Outlook) validan el registro DMARC, pero ante políticas nulas o inexistentes no tienen instrucción estricta de descarte, facilitando que los mensajes ingresen a la bandeja de entrada para engañar a los usuarios.

2. Explotación de software desactualizado en servidores web y proxies:

- **Objetivo principal:** Servidores con Apache 2.4.51 (unam.mx) y HAProxy 1.3–1.9 (hackthissite.org).
- **Vulnerabilidades en bases de datos oficiales:**
 - **CVE-2021-44790** [15] (NVD, CVSS 9.8 Crítico): “*A carefully crafted request body can cause a buffer overflow in the mod_lua multipart parser in Apache HTTP Server 2.4.51 and earlier*”. Un atacante podría enviar una petición HTTP POST especialmente construida con contenido multiparte para provocar la corrupción de memoria en `mod_lua`, permitiendo la ejecución remota de código o la caída inmediata del portal.
 - **CVE-2021-44224** [16] (NVD, CVSS 7.5 Alto): “*A crafted URI sent to httpd configured as a forward proxy can cause a crash (NULL pointer dereference)*”. Permite inducir condiciones de denegación de servicio (DoS).
 - **CVE-2019-18277** [20] (NVD, CVSS 7.5 Alto): “*HAProxy does not properly handle Transfer-Encoding and Content-Length headers, leading to HTTP Request Smuggling*”. Un atacante puede desincronizar las peticiones entre el proxy frontal y los servidores web de backend, logrando secuestrar sesiones de otros usuarios o envenenar la caché web.
- **Mecánica del ataque:** Mediante el envío de peticiones HTTP malformadas orientadas a los módulos vulnerables detectados en los escaneos de Nmap.

3. Infiltración y movimiento lateral a través de entornos de Desarrollo y QA:

- **Objetivos:** pemex.com (`api-cloud-des`, `ase3-digital-qa`) e ipn.mx (`servidor.dev`).
- **Incumplimiento técnico:** Incumplimiento de la guía metodológica OWASP WSTG-INFO-04 (Reconocimiento de aplicaciones de prueba y desarrollo) [13] y las pautas de segregación de redes de **NIST SP 800-115** [11].
- **Mecánica del ataque:** Los entornos de desarrollo suelen operar con versiones preliminares, certificados autofirmados, trazas de depuración activas y credenciales predeterminadas. Un atacante enfocaría sus esfuerzos en vulnerar estos subdominios secundarios para obtener secretos, cadenas de conexión o claves de API que posteriormente le permitan atacar los entornos productivos de la organización.

4. Secuestro de subdominios huérfanos (*Subdomain Takeover*):

- **Objetivo:** gob.mx (campañas, micrositos federales y recursos transitorios en la nube).
- **Incumplimiento técnico:** Violación a las directrices de verificación de configuración perimetral de **OWASP WSTG-CONF-10** (Pruebas de secuestro de subdominios) [13] y debilidad de control de acceso clasificada como **CWE-284** [19].
- **Mecánica del ataque:** Un atacante analiza la superficie masiva de subdominios de gob.mx identificando aquellos con registros DNS que apuntan a servicios en la nube externos (como AWS S3, Azure Web Apps o GitHub Pages) que han

sido dados de baja por las dependencias públicas pero cuyo registro DNS no fue eliminado. Al reclamar el nombre del recurso en dicho proveedor cloud, el atacante obtiene control total del subdominio gubernamental legítimo, pudiendo emitir certificados válidos, alojar portales falsos de recaudación o distribuir malware con el respaldo del dominio oficial `.gob.mx`.

5. Ingeniería social dirigida (*Spear Phishing*) contra el personal de infraestructura:

- **Objetivos:** Contactos administrativos y técnicos expuestos en WHOIS (`unam.mx`).
- **Fundamento:** Incumplimiento de las recomendaciones de minimización de datos en registros públicos del RFC 3912 (WHOIS) [7] y táctica de reconocimiento TA0043 de MITRE ATT&CK [14].
- **Mecánica del ataque:** Conociendo la identidad exacta de los responsables de la DGTIC, un atacante puede estructurar campañas de engaño telefónico o correos altamente personalizados [22], haciéndose pasar por proveedores de servicios o directivos para solicitar resets de contraseña o acceso a consolas de red.

2. Propuestas de mitigación y defensa (Recomendaciones de Seguridad)

Para remediar las debilidades descubiertas y robustecer la infraestructura, proponemos las siguientes medidas técnicas de mitigación:

1. Endurecimiento y transición obligatoria en DMARC (Remediación para UNAM, HackThisSite y gob.mx):

- Modificar la política DMARC en la UNAM y HackThisSite para migrar hacia **p=reject con pct=100**, siguiendo el estándar del IPN (`ipn.mx`).
- En `gob.mx`, publicar de forma prioritaria el registro DMARC en la zona DNS (`_dmarc.gob.mx`) con política de rechazo (**v=DMARC1; p=reject; rua=mailto:...**) para complementar su regla SPF actual y habilitar la recolección de reportes forenses ante cualquier intento de suplantación.
- Procesar de forma automatizada los reportes agregados enviados a las direcciones **rua** para detectar infraestructura no autorizada que intente emitir correo institucional.

2. Gestión de parches y ocultamiento de versiones (*Patch Management*):

- Actualizar de manera prioritaria los paquetes de Apache HTTP Server en los servidores de la UNAM a la rama estable más reciente (2.4.58 o superior), remediando los fallos **CVE-2021-44790** [15] y **CVE-2021-44224** [16]. De forma provisional, si el módulo `mod_lua` no es imprescindible, deshabilitarlo de la configuración de Apache.

- Actualizar HAProxy en hackthissite.org a una versión LTS moderna (2.8 o superior) para erradicar la susceptibilidad a *HTTP Request Smuggling* (CVE-2019-18277) [20].
 - Ocultar las cabeceras de versión en los servidores web (*Banner Grabbing Defense*): configurar `ServerTokens` `Prod` y `ServerSignature` `Off` en Apache, siguiendo el ejemplo implementado por Nginx en gob.mx para no regalar información valiosa a escaneos automáticos de reconocimiento.
- 3. Aislamiento de entornos de desarrollo, auditoría DNS y eliminación de Shadow IT:**
- **Retirar los subdominios de pruebas de la zona DNS pública:** Los subdominios de desarrollo (`dev`), pruebas (`qa`) y preproducción de Pemex y del IPN deben eliminarse del DNS público y trasladarse a zonas DNS internas privadas. El acceso a estos entornos debe condicionarse exclusivamente a través de una VPN corporativa con autenticación multifactor (MFA) [23, 24].
 - **Auditoría continua de registros CNAME y ciclo de vida (gob.mx):** Establecer una política centralizada de desaprovechamiento donde cualquier subdominio asignado a campañas o proveedores en la nube sea dado de baja en el DNS simultáneamente con el servicio en la nube, evitando el secuestro de subdominios.
 - **Auditoría y decomiso de sitios abandonados:** La UNAM debe realizar una depuración de subdominios para dar de baja páginas de congresos y eventos antiguos (como los de 2019), o bien convertirlas en páginas HTML puramente estáticas sin gestores de contenido (CMS) ni bases de datos activas en servidores desatendidos.
- 4. Aislamiento de cookies y protección de privacidad en WHOIS:**
- En plataformas gubernamentales masivas (gob.mx), restringir el alcance de las cookies de sesión para que no se declaren a nivel raíz con comodín (`Domain=.gob.mx`), previniendo ataques de fijación o captura de sesiones mediante subdominios comprometidos.
 - Sustituir los nombres completos y datos personales de los administradores en el registro de NIC México y LACNIC por cuentas de rol genéricas e institucionales (por ejemplo, `noc@unam.mx` o `seguridad-dns@unam.mx`), limitando la recolección pasiva de información (OSINT) [25].
 - Implementar programas periódicos de simulación de *phishing* y concientización en ciberseguridad dirigidos a los administradores y usuarios con privilegios elevados, reconociendo que la persona sigue siendo el eslabón más vulnerable de la seguridad institucional [22].

Preguntas

1. Menciona los tipos de protocolos de red.

Los protocolos de red son conjuntos de reglas y estándares formales que permiten a dos o más dispositivos comunicarse e intercambiar información a través de una red. Se pueden clasificar como sigue [26]:

- **Protocolos de comunicación:** Se encargan del direccionamiento lógico y del transporte de los datos entre dispositivos finales. Incluyen protocolos no orientados a conexión como IP (IPv4 e IPv6) y UDP, y protocolos orientados a conexión y confiables como TCP.
- **Protocolos de enrutamiento:** Descubren la topología de la red y calculan las rutas óptimas para reenviar los paquetes entre distintos sistemas autónomos o subredes (por ejemplo, BGP, OSPF y RIP).
- **Protocolos de gestión, control y soporte:** Permiten diagnosticar la conectividad, resolver direcciones físicas o asignar configuraciones dinámicas de red de forma automática.
- **Protocolos de seguridad:** Proveen mecanismos de cifrado, autenticación e integridad para proteger los datos en tránsito frente a interceptaciones y modificaciones.
- **Protocolos de aplicación:** Definen los formatos y reglas de intercambio para servicios específicos dirigidos al usuario final o a los procesos del sistema.

2. Respecto a la pregunta anterior. ¿Cómo funcionan? ¿Para qué sirven?

Respuesta:

¿Cómo funcionan? Los protocolos de red operan bajo el principio de encapsular por capas y el intercambio de Unidades de Datos de Protocolo. Cada capa añade una cabecera con metadatos de control. De acuerdo con Tanenbaum y Wetherall [26], todo protocolo se basa en tres componentes esenciales:

- Sintaxis:** Define la estructura y formato de los bloques de datos, indicando el orden exacto y la longitud en bits de cada campo dentro del paquete.
- Semántica:** Determina el significado específico de cada campo o patrón de bits, y las acciones o respuestas que cada host debe ejecutar al recibirlo (por ejemplo, el saludo de tres vías *SYN*, *SYN-ACK*, *ACK* en TCP).
- Temporización (Sincronización):** Establece el control de velocidad para la transmisión, la gestión de temporizadores (*timeouts*), el control de flujo y la re-transmisión de paquetes perdidos.

¿Para qué sirven? Buscan permitir la interoperabilidad global entre sistemas y dispositivos con diferente arquitectura, hardware y sistemas operativos [27]. En particular, sirven para:

- **Garantizar la entrega correcta de datos:** Detectar y corregir errores de transmisión, reordenar paquetes que llegan fuera de secuencia y regular la velocidad para no saturar los enlaces intermediarios.
- **Encaminar y multiplexar tráfico:** Dirigir los datos a través de rutas eficientes hacia su destino y asociar cada flujo de comunicación al proceso o aplicación local correcto mediante puertos lógicos.
- **Proteger la información:** Establecer canales cifrados y autenticados para mitigar ataques de intermediario, espionaje o suplantación de identidad.

3. ¿Qué es un sniffer?

Respuesta: Un *sniffer* es una herramienta de software o hardware diseñada para interceptar, capturar y decodificar el tráfico de datos que circula a través de una interfaz o segmento de red.

Para operar de forma completa, el sniffer suele colocar la tarjeta de interfaz de red (NIC) en **modo promiscuo**. En este modo, la tarjeta deja de descartar los paquetes que no van dirigidos a su propia dirección MAC y captura todas las tramas que circulan por el medio físico o inalámbrico compartido. Posteriormente, el software desensambla cada paquete capa por capa (desde el enlace de datos hasta la capa de aplicación), permitiendo inspeccionar las cabeceras y la carga útil (*payload*) [28].

4. ¿Qué significa OSINT? ¿Para qué sirve?

Respuesta: **OSINT** son las siglas en inglés de *Open Source Intelligence* (Inteligencia de Fuentes Abiertas). Hace referencia a la disciplina y metodología orientada a recopilar, procesar, correlacionar y analizar información legal y públicamente accesible para generar inteligencia accionable [25]. Abarca datos provenientes de registros públicos (WHOIS, DNS, certificados SSL), motores de búsqueda, redes sociales, repositorios de código, medios de comunicación y foros técnicos.

En el contexto de la ciberseguridad y las pruebas de penetración, OSINT sirve para:

- **Reconocimiento pasivo (*Footprinting*):** Permite mapear toda la infraestructura pública del objetivo (subdominios, direcciones IP, servicios en la nube y servidores de correo) sin interactuar directamente con sus servidores, evitando disparar alertas en los sistemas de detección de intrusos (IDS/IPS) o firewalls [11, 14].
- **Evaluación de la superficie de ataque:** Identifica tecnologías obsoletas, puertos potencialmente expuestos o paneles de acceso administrativo que no deberían estar a la vista de Internet.
- **Detección de fugas de datos y credenciales:** Localiza secretos empresariales, claves de API o contraseñas filtradas en repositorios públicos (como GitHub o GitLab) o en brechas de seguridad previas.
- **Análisis de perfiles y roles:** Permite conocer la estructura interna, organigrama y correos corporativos del personal para diseñar programas de capacitación y evaluar la susceptibilidad frente a campañas de ingeniería social.

5. Investiga los 5 OSINT más usados.

Respuesta: En el ámbito de la ciberseguridad, las investigaciones digitales y las pruebas de penetración, las cinco herramientas y plataformas de OSINT más utilizadas y reconocidas son:

- a) **Shodan:** Considerado el motor de búsqueda de dispositivos conectados a Internet por excelencia [29]. A diferencia de Google, que indexa páginas web, Shodan escanea e indexa los banners de respuesta de puertos abiertos, servidores web, cámaras de seguridad, bases de datos, dispositivos IoT y sistemas de control industrial (SCADA), revelando fallos de configuración y versiones expuestas.
- b) **Maltego:** Plataforma interactiva de minería de datos y análisis de grafos relacionales [30]. Permite transformar datos dispersos (nombres de personas, correos, dominios, IPs y redes sociales) en diagramas de red visuales que muestran de manera inmediata las conexiones y dependencias existentes entre diferentes entidades.
- c) **theHarvester:** Herramienta de consola ampliamente usada en la fase inicial de reconocimiento. Está diseñada para recolectar correos electrónicos de corporativos, nombres de colaboradores, subdominios, direcciones IP y URLs a partir de decenas de fuentes públicas como buscadores, servidores de claves PGP y bases de datos públicas.
- d) **SpiderFoot:** Plataforma de automatización OSINT con más de 200 módulos de recolección de datos integrados. Basta con proporcionar un objetivo (nombre de dominio, bloque de IPs, nombre de usuario o correo electrónico) para que realice un barrido integral de certificados, registros DNS, fugas de contraseñas y listas de reputación maliciosa.
- e) **Recon-ng:** Un entorno de trabajo (*framework*) modular y extensible con una interfaz de comandos muy similar a Metasploit, pero enfocado exclusivamente en inteligencia de fuentes abiertas. Cuenta con módulos independientes para consultar bases de datos públicas, APIs de servicios web y registros de infraestructura.

6. Investiga 5 softwares no mencionados en la práctica que sirvan para el análisis de comunicaciones.

Respuesta: Excluyendo las herramientas vistas en la práctica (`ping`, `nslookup`, `tracert`, `whois`, `sublist3r`, `subfinder`, `findomain`, `dnsmap`, `dnsrecon` y `nmap`), existen otros cinco softwares fundamentales para la captura, inspección y análisis de comunicaciones de red:

- a) **Wireshark:** Es el analizador de protocolos de red con interfaz gráfica más utilizado a nivel mundial [28]. Permite capturar el tráfico en tiempo real a nivel de interfaz de red, filtrar paquetes con precisión milimétrica mediante su potente lenguaje de filtros, reconstruir flujos completos de sesiones TCP/UDP e inspeccionar cabeceras en cientos de protocolos [31].

- b) **tcpdump:** La herramienta clásica por excelencia de línea de comandos para captura y análisis de paquetes basada en `libpcap`. Por su bajo consumo de recursos y ausencia de entorno gráfico, es el estándar indispensable para capturar tráfico en servidores remotos y guardar las trazas en archivos `.pcap` aplicando filtros de tipo BPF (*Berkeley Packet Filter*).
 - c) **Zeek (anteriormente Bro):** Un monitor y analizador de seguridad de red orientado a eventos y comportamiento [32]. A diferencia de un sniffer convencional que solo graba paquetes crudos, Zeek interpreta el tráfico y genera registros estructurados de alto nivel para transacciones web (HTTP), consultas DNS, certificados SSL/TLS y sesiones SSH.
 - d) **Bettercap:** Un entorno de trabajo completo y moderno para el análisis y auditoría de seguridad en redes cableadas e inalámbricas [33]. Permite realizar análisis de intermediario (*Man-in-the-Middle*), auditorías de tráfico ARP, DNS y DHCP en tiempo real, así como la inspección y manipulación de flujos de datos.
 - e) **Suricata:** Un motor de inspección profunda de paquetes (*Deep Packet Inspection*) de código abierto con arquitectura multihilo. Funciona tanto para el monitoreo de seguridad de red (NSM) como de sistema de detección y prevención de intrusiones (IDS/IPS), permitiendo analizar patrones de comunicación maliciosos a alta velocidad.
7. ¿Por qué se considera a las personas como el eslabón más débil de seguridad?

Respuesta: Las personas son consideradas el eslabón más débil en la cadena de seguridad porque, a diferencia de los sistemas computacionales y los algoritmos criptográficos que operan bajo reglas matemáticas predecibles y estrictas, el comportamiento humano está sujeto a emociones, sesgos cognitivos y descuidos involuntarios.

- **Susceptibilidad a la ingeniería social:** Los atacantes diseñan técnicas como el *phishing*, *pretexting* o la suplantación de identidad explotando rasgos psicológicos universales como la empatía, la curiosidad, el miedo, el respeto a la autoridad o la urgencia de resolver un problema.
- **Preferencia por la comodidad:** Los usuarios tienden a priorizar la conveniencia cotidiana sobre los protocolos de seguridad, lo que conduce a prácticas de alto riesgo como reutilizar la misma contraseña en múltiples servicios, compartir credenciales entre compañeros o ignorar avisos de certificados no válidos [24].
- **Fatiga y sobrecarga cognitiva:** El cansancio, las prisas o el exceso de alertas rutinarias disminuyen la atención del personal, aumentando la probabilidad de cometer errores humanos críticos.
- **Asimetría del ataque:** Para los equipos de seguridad, la defensa debe ser infalible en todo momento y para todos los empleados; para un atacante, basta con que un solo usuario caiga en un engaño para obtener un punto de apoyo inicial dentro de la red corporativa.

8. ¿Qué acciones haces para protegerte de ciberataques?

Respuesta: Previo a cursar lo poco que he cursado de la materia, no he aplicado muchas medidas para protegerme de ciberataques, sin embargo, algunas de las que utilizo son:

- **Autenticación multifactor (MFA/2FA):** Habilitar el doble factor de autenticación en todas las cuentas posibles, priorizando el uso de aplicaciones manejadoras de contraseñas seguras, y evitando la verificación por SMS debido al riesgo de suplantación de SIM.
- **Gestión de contraseñas seguras y únicas:** Emplear un gestor de contraseñas para crear y almacenar contraseñas aleatorias y complejas, erradicando por completo la reutilización de claves ante posibles filtraciones de datos.
- **Gestión de parches y actualizaciones periódicas:** Mantener actualizados el sistema operativo, los navegadores web y las aplicaciones para mitigar vulnerabilidades, realizo actualizaciones al menos uno o dos días después de salir la última versión, también para esperar a que salga información con respecto a la nueva actualización y asegurar que no se me exponga a vulnerabilidades de la actualización que puedan ser más graves.
- **Navegación precavida y verificación de enlaces:** Soy desconfiado frente a correos y mensajes con carácter urgente o sospechoso, me tomo mi tiempo para analizar el dominio remitente y la URL antes de abrir enlaces o descargar archivos adjuntos.
- **Copias de seguridad periódicas (estrategia 3-2-1):** Mantener respaldos frecuentes de los datos más importantes, incluyendo copias en almacenamiento local desconectado de la red (*offline*) para proteger la información contra ataques de secuestro de datos.

9. ¿Crees que tus métodos preventivos son suficientes?

Respuesta: No, no son 100 % suficientes, pues, como vimos en clase, en la ciberseguridad no existe la seguridad absoluta y considerarse totalmente a salvo es uno de los mayores errores que podemos cometer.

Como puede verse, la seguridad no es un producto ni un estado estático, sino un proceso continuo. Si bien las medidas preventivas cotidianas que utilizo logran mitigar la gran mayoría de los ataques oportunistas y automatizados, existen vectores de riesgo que escapan de mi control.

- **Vulnerabilidades de día cero (*Zero-Days*):** Fallos de seguridad en el sistema operativo, controladores o navegadores que son desconocidos para los desarrolladores y para los cuales aún no existe un parche disponible.
- **Ataques a la cadena de suministro (*Supply Chain Attacks*):** Situaciones en las que un atacante compromete una biblioteca de software, actualización legítima o dependencia antes de que llegue a los usuarios finales.

- **Filtraciones en servidores de terceros:** Aunque un usuario mantenga contraseñas únicas y robustas, la información personal sensible almacenada en servidores de empresas, universidades o entidades gubernamentales puede verse expuesta si dichas plataformas sufren una brecha de seguridad.
- **Acciones ajenas a mí en sistemas compartidos:** Dado que vivo con mi familia y me conecto a redes públicas, como la red de la facultad, no están en mis manos las acciones descuidadas que puedan tomar los miembros de mi familia o mis compañeros por el desconocimiento del peligro de sus acciones, exponiendo mi seguridad aunque tome las acciones de protección que mencioné antes.

Por ello, los estándares actuales de seguridad, tales como la Arquitectura de Confianza Cero (*Zero Trust*) del NIST [34], promueven el principio de **asumir la brecha** (*Assume Breach*). Esto significa que las medidas preventivas son indispensables para elevar el costo del ataque, pero deben complementarse siempre con una actitud vigilante, capacidad de detección temprana, contención de daños y políticas periódicas de respaldo y recuperación.

Referencias

- [1] National Institute of Standards and Technology (NIST), “National vulnerability database (NVD).” <https://nvd.nist.gov/>, 2026. Accedido: 17 de septiembre de 2026.
- [2] The MITRE Corporation, “Common vulnerabilities and exposures (CVE) list.” <https://www.cve.org/>, 2026. Accedido: 17 de septiembre de 2026.
- [3] Offensive Security, “Exploit database: Exploits, shellcode and vulnerability statistics.” <https://www.exploit-db.com/>, 2026. Accedido: 17 de septiembre de 2026.
- [4] Registro de Direcciones de Internet de América Latina y Caribe (LACNIC), “Servicios de registro y directorio de direccionamiento IP.” <https://www.lacnic.net/>, 2026. Accedido: 17 de septiembre de 2026.
- [5] Internet Assigned Numbers Authority (IANA), “Root zone database.” <https://www.iana.org/domains/root/db>, 2026. Accedido: 17 de septiembre de 2026.
- [6] NIC México, “Información y administración del dominio territorial de nivel superior .MX.” <https://www.nicmexico.mx/>, 2026. Accedido: 17 de septiembre de 2026.
- [7] L. Daigle, “WHOIS Protocol Specification.” RFC 3912, <https://datatracker.ietf.org/doc/html/rfc3912>, Sept. 2004. Accedido: 17 de septiembre de 2026.
- [8] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, “DNS Security Introduction and Requirements.” RFC 4033, <https://datatracker.ietf.org/doc/html/rfc4033>, Mar. 2005. Accedido: 17 de septiembre de 2026.

- [9] S. Kitterman, “Sender Policy Framework (SPF) for Authorizing Use of Domains in Email, Version 1.” RFC 7208, <https://datatracker.ietf.org/doc/html/rfc7208>, Apr. 2014. Accedido: 17 de septiembre de 2026.
- [10] M. Kucherawy and E. Zwicky, “Domain-based Message Authentication, Reporting, and Conformance (DMARC).” RFC 7489, <https://datatracker.ietf.org/doc/html/rfc7489>, Mar. 2015. Accedido: 17 de septiembre de 2026.
- [11] K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, “Technical guide to information security testing and assessment,” Tech. Rep. 800-115, National Institute of Standards and Technology (NIST), Sept. 2008. Accedido: 17 de septiembre de 2026.
- [12] PTES Team, “The penetration testing execution standard: Intelligence gathering.” http://www.pentest-standard.org/index.php/Intelligence_Gathering, 2014. Accedido: 17 de septiembre de 2026.
- [13] OWASP Foundation, “Web security testing guide (wstg) - v4.2: Information gathering.” <https://owasp.org/www-project-web-security-testing-guide/>, 2020. Accedido: 17 de septiembre de 2026.
- [14] The MITRE Corporation, “MITRE ATT&CK enterprise matrix: Tactic TA0043 (reconnaissance).” <https://attack.mitre.org/tactics/TA0043/>, 2024. Accedido: 17 de septiembre de 2026.
- [15] National Institute of Standards and Technology (NIST), “CVE-2021-44790 Detail: Apache HTTP Server mod_lua buffer overflow.” <https://nvd.nist.gov/vuln/detail/CVE-2021-44790>, 2021. Accedido: 17 de septiembre de 2026.
- [16] National Institute of Standards and Technology (NIST), “CVE-2021-44224 Detail: Apache HTTP Server mod_proxy NULL pointer dereference.” <https://nvd.nist.gov/vuln/detail/CVE-2021-44224>, 2021. Accedido: 17 de septiembre de 2026.
- [17] National Institute of Standards and Technology (NIST), “CVE-2022-1388 Detail: F5 BIG-IP iControl REST Authentication Bypass.” <https://nvd.nist.gov/vuln/detail/CVE-2022-1388>, 2022. Accedido: 17 de septiembre de 2026.
- [18] National Institute of Standards and Technology (NIST), “CVE-2023-46747 Detail: F5 BIG-IP Configuration Utility Unauthenticated Remote Code Execution.” <https://nvd.nist.gov/vuln/detail/CVE-2023-46747>, 2023. Accedido: 17 de septiembre de 2026.
- [19] The MITRE Corporation, “CWE-284: Improper Access Control.” <https://cwe.mitre.org/data/definitions/284.html>, 2024. Accedido: 17 de septiembre de 2026.
- [20] National Institute of Standards and Technology (NIST), “CVE-2019-18277 Detail: HAProxy HTTP Request Smuggling.” <https://nvd.nist.gov/vuln/detail/CVE-2019-18277>, 2019. Accedido: 17 de septiembre de 2026.

- [21] National Institute of Standards and Technology (NIST), “CVE-2020-11100 Detail: HAProxy HPACK Out-of-Bounds Memory Write and Buffer Overflow.” <https://nvd.nist.gov/vuln/detail/CVE-2020-11100>, 2020. Accedido: 17 de septiembre de 2026.
- [22] K. D. Mitnick and W. L. Simon, *The Art of Deception: Controlling the Human Element of Security*. John Wiley & Sons, 2002. Ficha editorial en: <https://www.wiley.com/en-us/The+Art+of+Deception%3A+Controlling+the+Human+Element+of+Security-p-9780764542800>.
- [23] Cybersecurity and Infrastructure Security Agency (CISA), “Secure our world: 4 easy ways to stay safe online.” <https://www.cisa.gov/secure-our-world>, 2024. Accedido: 17 de septiembre de 2026.
- [24] P. A. Grassi, J. L. Fenton, E. M. Newton, R. A. Perlner, A. R. Regenscheid, W. E. Burr, J. P. Richer, N. B. Lefkovitz, J. M. Danker, Y.-Y. Choong, K. K. Greene, and M. F. Theofanos, “Digital identity guidelines: Authentication and lifecycle management,” Tech. Rep. 800-63B, National Institute of Standards and Technology (NIST), 2020. Accedido: 17 de septiembre de 2026.
- [25] M. Bazzell, *Open Source Intelligence Techniques: Resources for Searching and Analyzing Online Information*. Independently published, 7ma ed., 2021. Página oficial y recursos: <https://inteltechniques.com/book1.html>.
- [26] A. S. Tanenbaum and D. J. Wetherall, *Redes de Computadoras*. Pearson Educación, 5ta ed., 2012. Disponible para consulta en línea: <https://archive.org/details/redesdecomputado0000tane>.
- [27] R. Braden, “Requirements for Internet Hosts - Communication Layers.” RFC 1122, <https://datatracker.ietf.org/doc/html/rfc1122>, Oct. 1989. Accedido: 17 de septiembre de 2026.
- [28] Wireshark Foundation, “Wireshark user’s guide.” https://www.wireshark.org/docs/wsug_html_chunked/, 2026. Accedido: 17 de septiembre de 2026.
- [29] J. Matherly, *Complete Guide to Shodan: Collect, Analyze, Visualize, Make Internet Intelligence Work for You*. Leanpub, 2016. Disponible en: <https://leanpub.com/shodan>.
- [30] Maltego Technologies, “Maltego: Link analysis and data mining for osint investigations.” <https://www.maltego.com/>, 2026. Accedido: 17 de septiembre de 2026.
- [31] C. Sanders, *Practical Packet Analysis: Using Wireshark to Solve Real-World Network Problems*. No Starch Press, 3ra ed., 2017. Información y recursos en: <https://nostarch.com/packetanalysis3>.
- [32] The Zeek Project, “Zeek: An open source network security monitor.” <https://zeek.org/>, 2026. Accedido: 17 de septiembre de 2026.

- [33] S. Margaritelli, “Bettercap: The swiss army knife for network attacks and monitoring.” <https://www.bettercap.org/>, 2026. Accedido: 17 de septiembre de 2026.
- [34] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” Tech. Rep. 800-207, National Institute of Standards and Technology (NIST), 2020. Accedido: 17 de septiembre de 2026.